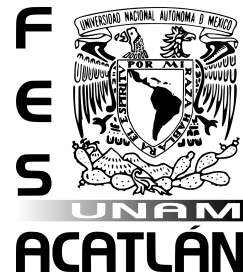


# UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

## FACULTAD DE ESTUDIOS SUPERIORES ACATLÁN



# REFERENCE

## Gatitas Tapabaños Nacatlán

by sav1209

Última actualización: 2026-03-30

# Contents

|                                                        |    |
|--------------------------------------------------------|----|
| 1. Setup .....                                         | 2  |
| 2. Data structures .....                               | 2  |
| 2.1. Prefix Sums .....                                 | 2  |
| 2.1.1. 2D Prefix Sums .....                            | 2  |
| 2.1.2. Max Subarray Sum .....                          | 2  |
| 2.2. Disjoin Set Union .....                           | 3  |
| 2.3. Segment tree .....                                | 4  |
| 3. Sorting & Searching .....                           | 4  |
| 3.1. Two Pointers .....                                | 4  |
| 3.2. Binary Search on Monotonic Functions .....        | 5  |
| 4. Bit Manipulation .....                              | 5  |
| 4.1. Bit mask .....                                    | 5  |
| 4.2. Representing Sets .....                           | 6  |
| 5. Graphs .....                                        | 7  |
| 5.1. Graph Traversal .....                             | 7  |
| 5.1.1. Floodfill .....                                 | 7  |
| 5.2. Topological Sort .....                            | 8  |
| 6. Number Theory .....                                 | 10 |
| 6.1. Primes and Factors .....                          | 10 |
| 6.2. Sieve of Eratosthenes .....                       | 10 |
| 6.3. Euclid's Algorithm .....                          | 10 |
| 6.3.1. Extended Euclid's Algorithm: .....              | 11 |
| 6.4. Euler's Theorem .....                             | 11 |
| 6.5. Solving Equations .....                           | 11 |
| 7. Combinatorics .....                                 | 12 |
| 7.1. Binomial Coefficients .....                       | 12 |
| 7.1.1. Properties .....                                | 12 |
| 7.1.2. Calculation .....                               | 12 |
| 7.1.2.1. Binomial coefficient modulo prime power ..... | 12 |
| 8. Utilities .....                                     | 13 |
| 8.1. Strings .....                                     | 13 |
| 8.1.1. Important Notes: .....                          | 14 |
| 8.1.2. Useful Patterns: .....                          | 14 |
| 8.2. STL Algorithm .....                               | 14 |
| 8.2.1. Important Notes: .....                          | 16 |
| 8.2.2. Useful Combinations: .....                      | 16 |
| 8.3. STL containers .....                              | 16 |
| 8.3.1. Deques .....                                    | 16 |
| 8.3.2. Set Structures .....                            | 16 |
| 8.3.2.1. Sets and Multisets .....                      | 16 |

|                                           |    |
|-------------------------------------------|----|
| 8.3.3. Priority Queues .....              | 17 |
| 8.3.4. Policy-based data structures ..... | 18 |

## 1. Setup

- To print  $n$  decimals: `cout << fixed << setprecision(n);`

### Bash script for compilation and execution

```
setup/compilation.sh ↗  
1  co() { g++ -std=c++20 -O2 -Wall -Wextra -Wshadow -Wconversion -o $1  
    $1.cpp; }  
2  
3  run() { co $1 && ./$1; }
```

## 2. Data structures

### 2.1. Prefix Sums

#### 2.1.1. 2D Prefix Sums

To process  $Q$  queries for the sum over a subrectangle of a 2D matrix with  $N$  rows and  $M$  columns

$$\text{prefix}[a][b] = \sum_{i=1}^a \sum_{j=1}^b \text{arr}[i][j]$$

This can be calculated as follows for row index  $1 \leq i \leq n$  and column index  $1 \leq j \leq m$ :

$$\text{prefix}[i][j] = \text{prefix}[i-1][j] + \text{prefix}[i][j-1] - \text{prefix}[i-1][j-1] + \text{arr}[i][j]$$

The submatrix sum between rows  $a$  and  $A$  and columns  $b$  and  $B$ , can thus be expressed as follows:

$$\sum_{i=a}^A \sum_{j=b}^B \text{arr}[i][j] = \text{prefix}[A][B] - \text{prefix}[a-1][B] - \text{prefix}[A][b-1] + \text{prefix}[a-1][b-1]$$

#### 2.1.2. Max Subarray Sum

Given an array of integers, find the maximum sum subarray

## Kadane's algorithm

TIME  
 $O(n)$

data\_structures/kadane.cpp ↗

```
6 int main() {
7     int n;
8     cin >> n;
9     vector<long long> x(n);
10    for (int i = 0; i < n; i++) { cin >> x[i]; }
11    ll current_sum = x[0];
12    ll max_subarray_sum = x[0];
13    for (int i = 1; i < n; i++) {
14        /*
15         * Continue the subarray sum or start a new
16         * subarray sum beginning at the current element.
17         */
18        current_sum = max(current_sum + x[i], x[i]);
19        // Store the maximum subarray sum at each iteration.
20        max_subarray_sum = max(max_subarray_sum, current_sum);
21    }
22    cout << max_subarray_sum << endl;
23 }
```

## 2.2. Disjoin Set Union

### Find representative with path compression

TIME  
 $O(\log n)$

data\_structures/dsu\_size.cpp ↗

```
11 int find(int v) {
12     if (v == parent[v])
13         return v;
14     return parent[v] = find(parent[v]);
15 }
```

### Union by size

TIME  
 $O(\alpha(m, n))$

data\_structures/dsu\_size.cpp ↗

```
17 DSU_size(int n) : parent(n), size(n, 1) {
18     for (int i = 0; i < n; i++) { parent[i] = i; }
19 }
20
21 bool unite(int a, int b) {
22     a = find(a);
23     b = find(b);
24
25     if (a == b) return false;
26
27     if (size[a] < size[b])
28         swap(a, b);
29     parent[b] = a;
30     size[a] += size[b];
31
32     return true;
33 }
```

### Union by rank

TIME  
 $O(\alpha(m, n))$

data\_structures/dsu\_rank.cpp ↗

```
17 DSU_rank(int n) : parent(n), rank(n, 0) {
18     for (int i = 0; i < n; i++) { parent[i] = i; }
19 }
20
21 bool unite(int a, int b) {
22     a = find(a);
23     b = find(b);
24
25     if (a == b) return false;
26
27     if (rank[a] < rank[b])
28         swap(a, b);
29     parent[b] = a;
30     if (rank[a] == rank[b])
31         rank[a]++;
32 }
```

```

33     return true;
34 }

```

## 2.3. Segment tree

### Implementation

- TIME
- $O(n)$  for building
  - $O(\log n)$  for queries and updates

data\_structures/segment-tree.cpp 

```

6  int n, t[4*MAXN];
7
8  void build(int a[], int v, int tl, int tr) {
9      if (tl == tr) {
10         t[v] = a[tl];
11     } else {
12         int tm = (tl + tr) / 2;
13         build(a, v*2, tl, tm);
14         build(a, v*2+1, tm+1, tr);
15         t[v] = t[v*2] + t[v*2+1];
16     }
17 }
18
19 int sum(int v, int tl, int tr, int l, int r) {
20     if (l > r)
21         return 0;
22     if (l == tl && r == tr) {
23         return t[v];
24     }
25     int tm = (tl + tr) / 2;
26     return sum(v*2, tl, tm, l, min(r, tm))
27         + sum(v*2+1, tm+1, tr, max(l, tm+1), r);
28 }
29
30 void update(int v, int tl, int tr, int pos, int new_val) {
31     if (tl == tr) {
32         t[v] = new_val;
33     } else {
34         int tm = (tl + tr) / 2;
35         if (pos <= tm)

```

```

36         update(v*2, tl, tm, pos, new_val);
37     else
38         update(v*2+1, tm+1, tr, pos, new_val);
39     t[v] = t[v*2] + t[v*2+1];
40 }
41 }

```

## 3. Sorting & Searching

### 3.1. Two Pointers

Some uses:

**Subarray sum:** Given an array of  $n$  positive integers and a target sum  $x$ , and we want to find a subarray whose sum is  $x$  or report that there is no such subarray.

**2SUM Problem:** Given an array of  $n$  numbers and a target sum  $x$ , find two array values such that their sum is  $x$ , or report that no such values exist.

**Merging two sorted arrays:** Given two arrays, sorted in ascending order combine the elements of these arrays into one big array.

```

1  while i < a.size() || j < b.size():
2      if a[i] < b[j]:
3          c[i + j] = a[i]
4          i++
5      else:
6          c[i + j] = b[j]
7          j++

```

**Number of Smaller:** We have two arrays  $a$  and  $b$ . We want to calculate for each element  $b_j$  how many such  $i$  exist that  $a_i < b_j$ .

```

1  i = 0
2  for j = 0..b.size()-1:
3      while i < a.size() && a[i] < b[j]:
4          i++
5      res[j] = i

```

## 3.2. Binary Search on Monotonic Functions

### Finding The Maximum $x$ Such That $f(x) = \text{true}$

TIME  
 $O(\log n)$

sorting\_searching/last\_true.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int last_true(int lo, int hi, function<bool(int)> f) {
5     // if none of the values in the range work, return lo - 1
6     lo--;
7     while (lo < hi) {
8         // find the middle of the current range (rounding up)
9         int mid = lo + (hi - lo + 1) / 2;
10        if (f(mid)) {
11            // if mid works, then all numbers smaller than mid also
            // work
12            lo = mid;
13        } else {
14            // if mid does not work, greater values would not work
            // either
15            hi = mid - 1;
16        }
17    }
18    return lo;
19 }
20
21 int main() {
22     // all numbers satisfy the condition (outputs 10)
23     cout << last_true(2, 10, [](int x) { return true; }) << endl;
24
25     // outputs 5
26     cout << last_true(2, 10, [](int x) { return x * x <= 30; }) <<
        endl;
27
28     // no numbers satisfy the condition (outputs 1)
29     cout << last_true(2, 10, [](int x) { return false; }) << endl;
30 }
```

### Finding The Minimum $x$ Such That $f(x) = \text{true}$

TIME  
 $O(\log n)$

sorting\_searching/first\_true.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int first_true(int lo, int hi, function<bool(int)> f) {
5     hi++;
6     while (lo < hi) {
7         int mid = lo + (hi - lo) / 2;
8         if (f(mid)) {
9             hi = mid;
10        } else {
11            lo = mid + 1;
12        }
13    }
14    return lo;
15 }
16
17 int main() {
18     // outputs 2
19     cout << first_true(2, 10, [](int x) { return true; }) << endl;
20     // outputs 6
21     cout << first_true(2, 10, [](int x) { return x * x >= 30; }) <<
        endl;
22     // outputs 11
23     cout << first_true(2, 10, [](int x) { return false; }) << endl;
24 }
```

## 4. Bit Manipulation

### 4.1. Bit mask

A *bit mask* of the form  $1 \ll k$  has a one bit in position  $k$  and all other bits are zero (zero-indexed from right to left). For **long long**, use  $1LL \ll k$ .

**Manipulating individual bits:**

$x \mid (1 \ll k)$ : Sets the  $k$ th bit of  $x$  to one.

$x \& \sim(1 \ll k)$ : Sets the  $k$ th bit of  $x$  to zero.

$x \oplus (1 \ll k)$ : Inverts the  $k$ th bit of  $x$ .

### Working with the rightmost 1-bit:

$x \& -x$ : Isolates the rightmost set bit (keeps only the least significant bit that is 1, sets all others to 0).

$x \& (x - 1)$ : Removes the rightmost set bit from  $x$  (turns the rightmost 1 to 0).

$x | (x - 1)$ : Sets all trailing zeros of  $x$  to 1.

### Creating useful bit masks:

$(1 \ll n) - 1$ : Creates a bit mask with the first  $n$  bits set to 1.

### Common bit manipulation tricks:

• A positive number  $x$  is a power of two exactly when  $x \& (x - 1) = 0$ .

$x \wedge A \wedge B$ : Determines which is not the current value of  $x$  between  $A$  and  $B$ .

### Bit operations as math alternatives:

$x \ll k$ : equivalent to  $x * \text{pow}(2, k)$

$x \gg k$ : equivalent to  $x / \text{pow}(2, k)$  (integer division)

$x \& ((1 \ll k) - 1)$ : equivalent to  $x \% \text{pow}(2, k)$

$A + B$ : equivalent to  $(A \wedge B) + 2 * (A \& B)$  or  $(A | B) - (A \& B)$

### GNU C++ compiler built-in functions:

`__builtin_clz(x)`: the number of leading zeros (zeros on the left side of the bit representation).

`__builtin_ctz(x)`: the number of trailing zeros (zeros on the right side of the bit representation).

`__builtin_popcount(x)`: the number of ones in the bit representation.

`__builtin_parity(x)`: the parity (even or odd) of the number of ones in the bit representation.

```
Example of use
1 int x = 5328; // 000000000000000000001010011010000
2 cout << __builtin_clz(x) << "\n"; // 19
3 cout << __builtin_ctz(x) << "\n"; // 4
4 cout << __builtin_popcount(x) << "\n"; // 5
5 cout << __builtin_parity(x) << "\n"; // 1
```

### Note

The above functions only support `int` numbers, but there are also `long long` versions of the functions available with the suffix `ll` (e.g.

`__builtin_popcountll(x)`).

## 4.2. Representing Sets

Every subset of a set  $\{0, 1, 2, \dots, n - 1\}$  can be represented as an  $n$  bit integer whose one bits indicate which elements belong to the subset.

*Example.* The bit representation of the set  $\{1, 3, 4, 8\}$  is

000000000000000000000000100011010

```
Prints the size of the set {1,3,4,8}
1 int x = 0;
2 x |= (1<<1);
3 x |= (1<<3);
4 x |= (1<<4);
5 x |= (1<<8);
6 cout << __builtin_popcount(x) << "\n"; // 4
```

```
Prints all elements that belong to the set
1 for (int i = 0; i < 32; i++) {
2     if (x&(1<<i)) cout << i << " ";
3 }
4 // output: 1 3 4 8
```

### Set operations:

| Operation    | Set syntax      | Bit syntax      |
|--------------|-----------------|-----------------|
| Intersection | $a \cap b$      | $a \& b$        |
| Union        | $a \cup b$      | $a   b$         |
| Complement   | $\bar{a}$       | $\sim a$        |
| Difference   | $a \setminus b$ | $a \& (\sim b)$ |

```
Goes through the subsets of {0,1,...,n-1}
1 for (int b = 0; b < (1<<n); b++) {
2     // process subset b
3 }
```

```
Goes through the subsets with exactly k elements
1 for (int b = 0; b < (1<<n); b++) {
2     if (__builtin_popcount(b) == k) {
3         // process subset b
4     }
5 }
```

#### Goes through the subsets of a set x



```
1 int b = 0;
2 do {
3     // process subset b
4 } while (b=(b-x)&x);
```

The idea is that the formula  $b - x$  detects the rightmost one bit in  $x$  that is zero in  $b$ . This bit becomes one and all bits after it become zero. Then the and operation ensures that the resulting value is a subset of  $x$ . Note that  $b - x$  equals  $-(x - b)$  so we can think that we first remove all one bits that appear in  $b$  and then invert the value and add one.

**C++ Bitsets:** The bitset structure corresponds to an array whose each value is either 0 or 1.

#### Creates a bitset of 10 elements



```
1 bitset<10> s;
2 s[1] = 1;
3 s[3] = 1;
4 s[4] = 1;
5 s[7] = 1;
6 cout << s[4] << "\n"; // 1
7 cout << s[5] << "\n"; // 0
```

#### Returns the number of one bits in the bitset



```
1 cout << s.count() << "\n"; // 4
```

#### Bit operations can be directly used to manipulate bitsets



```
1 bitset<10> a, b;
2 // ...
3 bitset<10> c = a&b;
4 bitset<10> d = a|b;
5 bitset<10> e = a^b;
```

## 5. Graphs

### 5.1. Graph Traversal

#### 5.1.1. Floodfill

Flood fill is an algorithm that identifies and labels the connected component that a particular cell belongs to in a multidimensional array.

#### Warning

Recursive implementations of flood fill sometimes lead to

- Memory limit exceeded if you run the recursive implementation on a really big grid (i.e., running the above code on a  $4000 \times 4000$  grid may exceed 256 MB)
  - Non-recursive implementations generally use less memory than recursive ones.

#### Example (recursive implementation)

##### graphs/floodfill\_recursive.cpp



```
1 const int MAX_N = 1000;
2
3 int grid[MAX_N][MAX_N]; // the grid itself
4 int row_num;
5 int col_num;
6 bool visited[MAX_N][MAX_N]; // keeps track of which nodes have been
    visited
7 int curr_size = 0; // reset to 0 each time we start a new
    component
8
9 void floodfill(int r, int c, int color) {
10     if ((r < 0 || r >= row_num || c < 0 || c >= col_num) // if out
        of bounds
11         || grid[r][c] != color // wrong
        color
12         || visited[r][c] // already
        visited this square
13     )
14         return;
15
16     visited[r][c] = true; // mark current square as visited
17     curr_size++; // increment the size for each square we
        visit
18
19     // recursively call flood fill for neighboring squares
20     floodfill(r, c + 1, color);
21     floodfill(r, c - 1, color);
22     floodfill(r - 1, c, color);
23     floodfill(r + 1, c, color);
24 }
```

```

25
26 int main() {
27     /*
28      * input code and other problem-specific stuff here
29      */
30     for (int i = 0; i < row_num; i++) {
31         for (int j = 0; j < col_num; j++) {
32             if (!visited[i][j]) {
33                 curr_size = 0;
34                 /*
35                  * start a flood fill if the square hasn't already
36                  * and then store or otherwise use the component size
37                  * for whatever it's needed for
38                  */
39                 floodfill(i, j, grid[i][j]);
40             }
41         }
42     }
43     return 0;
44 }

```

### Example (iterative implementation)

graphs/floodfill\_iterative.cpp 

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  const int MAX_N = 2500;
6  const int R_CHANGE[] {0, 1, 0, -1};
7  const int C_CHANGE[] {1, 0, -1, 0};
8
9  int row_num;
10 int col_num;
11 string building[MAX_N];
12 bool visited[MAX_N][MAX_N];
13
14 void floodfill(int r, int c) {
15     // Note: you can also use a queue and pop from the front for a
16     // BFS-based

```

```

16     // approach
17     stack<pair<int, int>> frontier;
18     frontier.push({r, c});
19     while (!frontier.empty()) {
20         r = frontier.top().first;
21         c = frontier.top().second;
22         frontier.pop();
23
24         if (r < 0 || r >= row_num || c < 0 || c >= col_num ||
25             building[r][c] == '#' ||
26             visited[r][c])
27             continue;
28         visited[r][c] = true;
29         for (int i = 0; i < 4; i++) {
30             frontier.push({r + R_CHANGE[i], c + C_CHANGE[i]});
31         }
32     }
33 }
34
35 int main() {
36     cin >> row_num >> col_num;
37     for (int i = 0; i < row_num; i++) { cin >> building[i]; }
38
39     int room_num = 0;
40     for (int i = 0; i < row_num; i++) {
41         for (int j = 0; j < col_num; j++) {
42             if (building[i][j] == '.' && !visited[i][j]) {
43                 floodfill(i, j);
44                 room_num++;
45             }
46         }
47     }
48     cout << room_num << endl;
49 }

```

## 5.2. Topological Sort

A topological sort of a directed acyclic graph is a linear ordering of its vertices such that for every directed edge  $u \rightarrow v$  from vertex  $u$  to vertex  $v$ ,  $u$  comes before  $v$  in the ordering.



## DFS Version

graphs/topological\_sort\_dfs.cpp 

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  vector<int> top_sort;
6  vector<vector<int>> graph;
7  vector<bool> visited;
8
9  void dfs(int node) {
10     for (int next : graph[node]) {
11         if (!visited[next]) {
12             visited[next] = true;
13             dfs(next);
14         }
15     }
16     top_sort.push_back(node);
17 }
18
19 int main() {
20     int n, m; // The number of nodes and edges respectively
21     cin >> n >> m;
22
23     graph = vector<vector<int>>(n);
24     for (int i = 0; i < m; i++) {
25         int a, b;
26         cin >> a >> b;
27         graph[a - 1].push_back(b - 1);
28     }
29
30     visited = vector<bool>(n);
31     for (int i = 0; i < n; i++) {
32         if (!visited[i]) {
33             visited[i] = true;
34             dfs(i);
35         }
36     }
37     reverse(top_sort.begin(), top_sort.end());
38 }
```

```
39     vector<int> ind(n);
40     for (int i = 0; i < n; i++) { ind[top_sort[i]] = i; }
41
42     // Check if the topological sort is valid
43     bool valid = true;
44     for (int i = 0; i < n; i++) {
45         for (int j : graph[i]) {
46             if (ind[j] <= ind[i]) {
47                 valid = false;
48                 goto answer;
49             }
50         }
51     }
52     answer;
53
54     if (valid) {
55         for (int i = 0; i < n - 1; i++) { cout << top_sort[i] + 1 <<
56             ' '; }
57         cout << top_sort.back() + 1 << endl;
58     } else {
59         cout << "IMPOSSIBLE" << endl;
60     }
```

## BFS Version (Kahn's Algorithm)

graphs/topological\_sort\_bfs.cpp 

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6     int n, m;
7     cin >> n >> m;
8
9     vector<vector<int>> graph(n);
10     for (int i = 0; i < m; i++) {
11         int a, b;
12         cin >> a >> b;
13         graph[a - 1].push_back(b - 1);
14     }
```

```

15
16 vector<int> in_degree(n);
17 for (const vector<int> &nodes : graph) {
18     for (int node : nodes) { in_degree[node]++; }
19 }
20
21 queue<int> queue;
22 for (int i = 0; i < n; i++) {
23     if (in_degree[i] == 0) { queue.push(i); }
24 }
25 vector<int> top_sort;
26 while (!queue.empty()) {
27     int curr = queue.front();
28     queue.pop();
29     top_sort.push_back(curr);
30     for (int next : graph[curr]) {
31         if (--in_degree[next] == 0) { queue.push(next); }
32     }
33 }
34
35 if (top_sort.size() == n) {
36     for (int i = 0; i < n - 1; i++) { cout << top_sort[i] + 1 <<
37         ' '; }
38     cout << top_sort.back() + 1 << endl;
39 } else {
40     cout << "IMPOSSIBLE" << endl;
41 }

```

## 6. Number Theory

The *prime-counting function*  $\pi(n)$  gives the number of primes up to  $n$ .

$$\pi(n) \approx \frac{n}{\ln(n)}$$

### 6.1. Primes and Factors

For every integer  $n > 1$ , there is a unique prime factorization

$$n = p_1^{\alpha_1} p_2^{\alpha_2} \dots p_k^{\alpha_k}$$

Number of factors of an integer  $n$

$$\tau(n) = \prod_{i=1}^k (\alpha_i + 1)$$

Sum of factors of an integer  $n$

$$\sigma(n) = \prod_{i=1}^k (1 + p_i + \dots + p_i^{\alpha_i}) = \prod_{i=1}^k \frac{p_i^{\alpha_i+1} - 1}{p_i - 1}$$

### 6.2. Sieve of Eratosthenes

#### Sieve with basic optimizations

|                            |                 |
|----------------------------|-----------------|
| TIME<br>$O(n \log \log n)$ | SPACE<br>$O(n)$ |
|----------------------------|-----------------|

Find all the prime numbers in  $[1, n]$ . This code uses the next optimizations:

- Sieving till root
- Sieving by the odd numbers only

#### number\_theory/sieve\_with\_basic\_optimizations.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<bool> sieve(int n) {
5     vector<bool> is_prime(n + 1, true);
6     is_prime[0] = is_prime[1] = false;
7     for (int i = 3; i * i ≤ n; i += 2) {
8         if (is_prime[i]) {
9             for (int j = i * i; j ≤ n; j += 2 * i)
10                 is_prime[j] = false;
11         }
12     }
13     return is_prime;
14 }

```

### 6.3. Euclid's Algorithm

The algorithm is based on the formula

$$\gcd(a, b) = \begin{cases} a & b = 0 \\ \gcd(b, a \bmod b) & b \neq 0 \end{cases}$$

### 6.3.1. Extended Euclid's Algorithm:

Euclid's algorithm can also be extended so that it gives integers  $x$  and  $y$  for which

$$ax + by = \gcd(a, b)$$

#### Iterative version

TIME  
 $O(\log \min(a, b))$

number\_theory/extended\_euclid\_algorithm.cpp

```
1 int gcd(int a, int b, int& x, int& y) {
2     x = 1, y = 0;
3     int x1 = 0, y1 = 1, a1 = a, b1 = b;
4     while (b1) {
5         int q = a1 / b1;
6         tie(x, x1) = make_tuple(x1, x - q * x1);
7         tie(y, y1) = make_tuple(y1, y - q * y1);
8         tie(a1, b1) = make_tuple(b1, a1 - q * b1);
9     }
10    return a1;
11 }
```

## 6.4. Euler's Theorem

Euler's totient function  $\varphi(n)$  gives the number of integers between  $1, \dots, n$  that are coprime to  $n$ .

Any value of  $\varphi(n)$  can be calculated from the prime factorization of  $n$  using the formula

$$\varphi(n) = \prod_{i=1}^k p_i^{\alpha_i-1} (p_i - 1)$$

#### Theorem 6.4.1 (Euler's theorem)

For all positive coprime integers  $x$  and  $m$

$$x^{\varphi(m)} \bmod m = 1$$

If  $m$  is prime,  $\varphi(m) = m - 1$ , so the formula becomes

$$x^{m-1} \bmod m = 1$$

which is known as *Fermat's little theorem*. This also implies that

$$x^n \bmod m = x^{n \bmod (m-1)} \bmod m$$

which can be used to calculate values of  $x^n$  if  $n$  is very large.

#### Definition 6.4.2 (Modular multiplicative inverse)

The *modular multiplicative inverse* of  $x$  with respect to  $m$  is a value  $\text{inv}_m(x)$  such that

$$x \cdot \text{inv}_m(x) \bmod m = 1$$

A modular multiplicative inverse exists exactly when  $x$  and  $m$  are coprime. In this case it can be calculated using the formula

$$\text{inv}_m(x) = x^{\varphi(m)-1}$$

In particular, if  $m$  is prime,  $\varphi(m) = m - 1$  and the formula becomes

$$\text{inv}_m(x) = x^{m-2}$$

The above formula allows us to efficiently calculate modular multiplicative inverses using the modular exponentiation algorithm.

## 6.5. Solving Equations

#### Definition 6.5.1 (Diophantine equation)

A *Diophantine equation* is an equation of the form

$$ax + by = c$$

where  $a, b$  and  $c$  are constants and the values of  $x$  and  $y$  should be found. Each number in the equation has to be an integer.

We can efficiently solve a Diophantine equation by using the extended Euclid's algorithm which gives integers  $x$  and  $y$  that satisfy the equation

$$ax + by = \gcd(a, b)$$

A Diophantine equation can be solved exactly when  $c$  is divisible by  $\gcd(a, b)$ .

If a pair  $(x, y)$  is a solution, then also all pairs

$$\left( x + \frac{kb}{\gcd(a, b)}, y - \frac{ka}{\gcd(a, b)} \right)$$

are solutions, where  $k$  is any integer.

# 7. Combinatorics

## 7.1. Binomial Coefficients

### Note

For  $n < k$  the value of  $\binom{n}{k}$  is assumed to be zero.

### 7.1.1. Properties

Pascal's Triangle:

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

Symmetry rule:

$$\binom{n}{k} = \binom{n}{n-k}$$

Factoring in:

$$\binom{n}{k} = \frac{n}{k} \binom{n-1}{k-1}$$

Sum over  $k$ :

$$\sum_{k=0}^n \binom{n}{k} = 2^n$$

Sum over  $n$ :

$$\sum_{m=0}^n \binom{m}{k} = \binom{n+1}{k+1}$$

Sum over  $n$  and  $k$ :

$$\sum_{k=0}^m \binom{n+k}{k} = \binom{n+m+1}{m}$$

Sum of the squares:

$$\binom{n}{0}^2 + \binom{n}{1}^2 + \dots + \binom{n}{n}^2 = \binom{2n}{n}$$

Weighted sum:

$$1\binom{n}{1} + 2\binom{n}{2} + \dots + n\binom{n}{n} = n2^{n-1}$$

Connection with the Fibonacci numbers:

$$\binom{n}{0} + \binom{n-1}{1} + \dots + \binom{n-k}{k} + \dots + \binom{0}{n} = F_{n+1}$$

### 7.1.2. Calculation

#### Basic Improved Implementation

```
1 int C(int n, int k) {
2     double res = 1;
3     for (int i = 1; i ≤ k; ++i)
4         res = res * (n - k + i) / i;
5     return (int)(res + 0.01);
6 }
```

#### Table of binomial coefficients using Pascal's Triangle identity

```
1 const int maxn = ... ;
2 int C[maxn + 1][maxn + 1];
3 C[0][0] = 1;
4 for (int n = 1; n ≤ maxn; ++n) {
5     C[n][0] = C[n][n] = 1;
6     for (int k = 1; k < n; ++k)
7         C[n][k] = C[n - 1][k - 1] + C[n - 1][k];
8 }
```

### 7.1.2.1. Binomial coefficient modulo prime power

$$\binom{n}{k} \bmod p \equiv (n!) \operatorname{inv}_p((n-k)!) \operatorname{inv}_p(k!) \bmod p$$

$$\operatorname{inv}_p((x-1)!) \equiv x * \operatorname{inv}_p(x!) \bmod p$$

# 8. Utilities

## 8.1. Strings

| Method/Syntax                                                       | Description                                 | Complexity       | Example                                    |
|---------------------------------------------------------------------|---------------------------------------------|------------------|--------------------------------------------|
| <code>s.size()</code> / <code>s.length()</code>                     | Returns the number of characters            | $O(1)$           | <code>int n = s.size();</code>             |
| <code>s.empty()</code>                                              | Checks if string is empty                   | $O(1)$           | <code>if (s.empty()) { ... }</code>        |
| <code>s[i]</code> / <code>s.at(i)</code>                            | Access character at position i              | $O(1)$           | <code>char c = s[0];</code>                |
| <code>s.front()</code> / <code>s.back()</code>                      | Access first/last character                 | $O(1)$           | <code>char first = s.front();</code>       |
| <code>s.push_back(c)</code>                                         | Appends character to end                    | $O(1)$ amortized | <code>s.push_back('x');</code>             |
| <code>s.pop_back()</code>                                           | Removes last character                      | $O(1)$           | <code>s.pop_back();</code>                 |
| <code>s += str</code> / <code>s.append(str)</code>                  | Concatenates string/character               | $O(m)$           | <code>s += "hello";</code>                 |
| <code>s.insert(pos, str)</code>                                     | Inserts string at position                  | $O(n+m)$         | <code>s.insert(2, "abc");</code>           |
| <code>s.erase(pos, len)</code>                                      | Removes len characters from pos             | $O(n)$           | <code>s.erase(2, 3);</code>                |
| <code>s.substr(pos, len)</code>                                     | Returns substring from pos with len chars   | $O(len)$         | <code>string sub = s.substr(2, 5);</code>  |
| <code>s.find(str, pos)</code>                                       | Finds first occurrence of str from pos      | $O(n*m)$         | <code>size_t pos = s.find("abc");</code>   |
| <code>s.rfind(str, pos)</code>                                      | Finds last occurrence of str before pos     | $O(n*m)$         | <code>size_t pos = s.rfind("abc");</code>  |
| <code>s.find_first_of(chars)</code>                                 | Finds first occurrence of any char in chars | $O(n*m)$         | <code>s.find_first_of("aeiou");</code>     |
| <code>s.find_last_of(chars)</code>                                  | Finds last occurrence of any char in chars  | $O(n*m)$         | <code>s.find_last_of("aeiou");</code>      |
| <code>s.find_first_not_of(chars)</code>                             | Finds first char not in chars               | $O(n*m)$         | <code>s.find_first_not_of(" \t");</code>   |
| <code>s.find_last_not_of(chars)</code>                              | Finds last char not in chars                | $O(n*m)$         | <code>s.find_last_not_of(" \t");</code>    |
| <code>s.replace(pos, len, str)</code>                               | Replaces len chars at pos with str          | $O(n+m)$         | <code>s.replace(2, 3, "xyz");</code>       |
| <code>s.compare(str)</code>                                         | Lexicographically compares strings          | $O(\min(n,m))$   | <code>int cmp = s.compare("abc");</code>   |
| <code>s.c_str()</code>                                              | Returns C-style null-terminated string      | $O(1)$           | <code>const char* cstr = s.c_str();</code> |
| <code>s.data()</code>                                               | Returns pointer to character array          | $O(1)$           | <code>char* ptr = s.data();</code>         |
| <code>s.reserve(n)</code>                                           | Reserves capacity for n characters          | $O(1)$           | <code>s.reserve(1000);</code>              |
| <code>s.resize(n, c)</code>                                         | Changes size to n, filling with c if needed | $O(n)$           | <code>s.resize(10, 'x');</code>            |
| <code>s.clear()</code>                                              | Removes all characters                      | $O(n)$           | <code>s.clear();</code>                    |
| <code>s.swap(other)</code>                                          | Swaps contents with another string          | $O(1)$           | <code>s.swap(t);</code>                    |
| <code>getline(cin, s)</code>                                        | Reads entire line including spaces          | $O(n)$           | <code>getline(cin, s);</code>              |
| <code>getline(cin, s, delim)</code>                                 | Reads until delimiter character             | $O(n)$           | <code>getline(cin, s, ',');</code>         |
| <code>stoi(s)</code> / <code>stol(s)</code> / <code>stoll(s)</code> | Converts string to int/long/long long       | $O(n)$           | <code>int num = stoi("123");</code>        |
| <code>stof(s)</code> / <code>stod(s)</code>                         | Converts string to float/double             | $O(n)$           | <code>double d = stod("3.14");</code>      |

|                                                   |                                       |                      |                                         |
|---------------------------------------------------|---------------------------------------|----------------------|-----------------------------------------|
| <code>to_string(num)</code>                       | Converts number to string             | $O(\log \text{num})$ | <code>string s = to_string(42);</code>  |
| <code>isalpha(c)</code> / <code>isdigit(c)</code> | Checks if char is letter/digit        | $O(1)$               | <code>if (isalpha(s[i])) { ... }</code> |
| <code>islower(c)</code> / <code>isupper(c)</code> | Checks if char is lowercase/uppercase | $O(1)$               | <code>if (islower(s[i])) { ... }</code> |
| <code>tolower(c)</code> / <code>toupper(c)</code> | Converts char to lowercase/uppercase  | $O(1)$               | <code>s[i] = tolower(s[i]);</code>      |
| <code>isspace(c)</code>                           | Checks if char is whitespace          | $O(1)$               | <code>if (isspace(s[i])) { ... }</code> |
| <code>isalnum(c)</code>                           | Checks if char is alphanumeric        | $O(1)$               | <code>if (isalnum(s[i])) { ... }</code> |

### 8.1.1. Important Notes:

- **String indexing:** Use `s[i]` for fast access, `s.at(i)` for bounds checking
- **Find functions:** Return `string::npos` if not found (usually -1 or large value)
- **Character functions:** `isalpha`, `isdigit`, etc. are in `<cctype>` header
- **Conversion functions:** `stoi`, `to_string`, etc. are in `<string>` header
- **Amortized complexity:** `push_back` is  $O(1)$  on average due to capacity doubling

### 8.1.2. Useful Patterns:

```

1 // Check if substring exists
2 if (s.find("pattern") != string::npos) { ... }
3
4 // Convert string to lowercase
5 transform(s.begin(), s.end(), s.begin(), [](char c) { return tolower(c); });
6
7 // Split string by delimiter
8 stringstream ss(s); string token;
9 while (getline(ss, token, ',')) { ... }
10
11 // Remove leading/trailing whitespace
12 s.erase(0, s.find_first_not_of(" \t"));
13 s.erase(s.find_last_not_of(" \t") + 1);
14
15 // Reverse string
16 reverse(s.begin(), s.end());

```

## 8.2. STL Algorithm

| Function/Syntax                      | Description                                        | Complexity    | Example                                      |
|--------------------------------------|----------------------------------------------------|---------------|----------------------------------------------|
| <code>sort(begin, end)</code>        | Sorts elements in ascending order                  | $O(n \log n)$ | <code>sort(v.begin(), v.end())</code>        |
| <code>stable_sort(begin, end)</code> | Sorts maintaining relative order of equal elements | $O(n \log n)$ | <code>stable_sort(v.begin(), v.end())</code> |

|                                                                |                                                        |            |                                                                                  |
|----------------------------------------------------------------|--------------------------------------------------------|------------|----------------------------------------------------------------------------------|
| <code>reverse(begin, end)</code>                               | Reverses the order of elements                         | O(n)       | <code>reverse(v.begin(), v.end())</code>                                         |
| <code>binary_search(begin, end, val)</code>                    | Searches for value in sorted range (returns bool)      | O(log n)   | <code>binary_search(v.begin(), v.end(), x)</code>                                |
| <code>lower_bound(begin, end, val)</code>                      | Finds first element $\geq$ val                         | O(log n)   | <code>lower_bound(v.begin(), v.end(), x)</code>                                  |
| <code>upper_bound(begin, end, val)</code>                      | Finds first element $>$ val                            | O(log n)   | <code>upper_bound(v.begin(), v.end(), x)</code>                                  |
| <code>equal_range(begin, end, val)</code>                      | Returns pair of iterators [lower_bound, upper_bound]   | O(log n)   | <code>equal_range(v.begin(), v.end(), x)</code>                                  |
| <code>min_element(begin, end)</code>                           | Finds iterator to minimum element                      | O(n)       | <code>min_element(v.begin(), v.end())</code>                                     |
| <code>max_element(begin, end)</code>                           | Finds iterator to maximum element                      | O(n)       | <code>max_element(v.begin(), v.end())</code>                                     |
| <code>minmax_element(begin, end)</code>                        | Returns pair {min_iter, max_iter}                      | O(n)       | <code>minmax_element(v.begin(), v.end())</code>                                  |
| <code>next_permutation(begin, end)</code>                      | Generates next lexicographic permutation               | O(n)       | <code>next_permutation(v.begin(), v.end())</code>                                |
| <code>prev_permutation(begin, end)</code>                      | Generates previous lexicographic permutation           | O(n)       | <code>prev_permutation(v.begin(), v.end())</code>                                |
| <code>unique(begin, end)</code>                                | Removes consecutive duplicate elements                 | O(n)       | <code>unique(v.begin(), v.end())</code>                                          |
| <code>count(begin, end, val)</code>                            | Counts occurrences of a value                          | O(n)       | <code>count(v.begin(), v.end(), x)</code>                                        |
| <code>count_if(begin, end, pred)</code>                        | Counts elements satisfying condition                   | O(n)       | <code>count_if(v.begin(), v.end(), [](int x){return x%2==0;})</code>             |
| <code>find(begin, end, val)</code>                             | Finds first occurrence of value                        | O(n)       | <code>find(v.begin(), v.end(), x)</code>                                         |
| <code>find_if(begin, end, pred)</code>                         | Finds first element satisfying condition               | O(n)       | <code>find_if(v.begin(), v.end(), [](int x){return x&gt;5;})</code>              |
| <code>rotate(begin, middle, end)</code>                        | Rotates elements (middle becomes new begin)            | O(n)       | <code>rotate(v.begin(), v.begin()+k, v.end())</code>                             |
| <code>random_shuffle(begin, end)</code>                        | Shuffles elements randomly                             | O(n)       | <code>random_shuffle(v.begin(), v.end())</code>                                  |
| <code>shuffle(begin, end, rng)</code>                          | Shuffles with specific random generator                | O(n)       | <code>shuffle(v.begin(), v.end(), rng)</code>                                    |
| <code>fill(begin, end, val)</code>                             | Fills range with a value                               | O(n)       | <code>fill(v.begin(), v.end(), 0)</code>                                         |
| <code>iota(begin, end, val)</code>                             | Fills with consecutive values starting from val        | O(n)       | <code>iota(v.begin(), v.end(), 1)</code>                                         |
| <code>accumulate(begin, end, init)</code>                      | Sums elements (requires <code>&lt;numeric&gt;</code> ) | O(n)       | <code>accumulate(v.begin(), v.end(), 0)</code>                                   |
| <code>partial_sum(begin, end, out)</code>                      | Calculates partial sums                                | O(n)       | <code>partial_sum(v.begin(), v.end(), prefix.begin())</code>                     |
| <code>merge(begin1, end1, begin2, end2, out)</code>            | Merges two sorted ranges                               | O(n+m)     | <code>merge(a.begin(), a.end(), b.begin(), b.end(), c.begin())</code>            |
| <code>inplace_merge(begin, middle, end)</code>                 | Merges two sorted parts of same range                  | O(n log n) | <code>inplace_merge(v.begin(), v.begin()+k, v.end())</code>                      |
| <code>set_union(begin1, end1, begin2, end2, out)</code>        | Union of two sorted sets                               | O(n+m)     | <code>set_union(a.begin(), a.end(), b.begin(), b.end(), c.begin())</code>        |
| <code>set_intersection(begin1, end1, begin2, end2, out)</code> | Intersection of two sorted sets                        | O(n+m)     | <code>set_intersection(a.begin(), a.end(), b.begin(), b.end(), c.begin())</code> |
| <code>set_difference(begin1, end1, begin2, end2, out)</code>   | Difference of two sorted sets                          | O(n+m)     | <code>set_difference(a.begin(), a.end(), b.begin(), b.end(), c.begin())</code>   |

|                                                   |                                                  |            |                                                                        |
|---------------------------------------------------|--------------------------------------------------|------------|------------------------------------------------------------------------|
| <code>nth_element(begin, nth, end)</code>         | Partitions so nth element is in correct position | $O(n)$ avg | <code>nth_element(v.begin(), v.begin()+k, v.end())</code>              |
| <code>partition(begin, end, pred)</code>          | Partitions elements by predicate                 | $O(n)$     | <code>partition(v.begin(), v.end(), [](int x) {return x%2==0;})</code> |
| <code>is_sorted(begin, end)</code>                | Checks if range is sorted                        | $O(n)$     | <code>is_sorted(v.begin(), v.end())</code>                             |
| <code>is_permutation(begin1, end1, begin2)</code> | Checks if one range is permutation of another    | $O(n^2)$   | <code>is_permutation(a.begin(), a.end(), b.begin())</code>             |

### 8.2.1. Important Notes:

- **Sorted ranges required:** `binary_search`, `lower_bound`, `upper_bound`, `equal_range`, `merge`, `set_*` require sorted ranges
- **<numeric> functions:** `accumulate`, `partial_sum`, `iota` are in `<numeric>`, not `<algorithm>`
- **Iterators:** Many functions return iterators. To get indices: `distance(v.begin(), iter)`
- **Predicates:** Functions like `count_if`, `find_if` accept lambdas or functors
- **Custom comparators:** Most accept custom comparators as third parameter

### 8.2.2. Useful Combinations:

```
1 // Sort and remove duplicates
2 sort(v.begin(), v.end());
3 v.erase(unique(v.begin(), v.end()), v.end());
```

## 8.3. STL containers

### 8.3.1. Deques

A **deque** is a dynamic array that can be efficiently manipulated at both ends of the structure.

```
1 deque<int> d;
2 d.push_back(5); // [5]
3 d.push_back(2); // [5,2]
4 d.push_front(3); // [3,5,2]
5 d.pop_back(); // [3,5]
6 d.pop_front(); // [5]
```

The operations of a deque also work in  $O(1)$  average time. Deques should be used only if there is a need to manipulate both ends of the array.

### 8.3.2. Set Structures

The basic operations of sets are element insertion, search and removal. Sets are implemented so that all the above operations are efficient.

#### 8.3.2.1. Sets and Multisets

##### Basic functions

- `insert` adds an element to the set
- `count` returns the number of occurrences of an element in the set
- `erase` removes an element from the set
- `find(x)` returns an iterator that points to an element whose value is `x`. However, if the set does not contain `x`, the iterator will be `end()`.



Prints the number of elements in a set

```
1 cout << s.size() << "\n";
2 for (auto x : s) {
3     cout << x << "\n";
4 }
```

Use of find

```
1 auto it = s.find(x);
2 if (it == s.end()) {
3     // x is not found
4 }
```

**Multisets:** A *multiset* is a set that can have several copies of the same value. C++ has the structures `multiset` and `unordered_multiset`.

```
1 multiset<int> s;
2 s.insert(5);
3 s.insert(5);
4 s.insert(5);
5 cout << s.count(5) << "\n"; // 3
```

The function `erase` removes all copies of a value from a multiset:

```
1 s.erase(5);
2 cout << s.count(5) << "\n"; // 0
```

Often, only one value should be removed, which can be done as follows:

```
1 s.erase(s.find(5));
2 cout << s.count(5) << "\n"; // 2
```

Note that the functions `count` and `erase` have an additional  $O(k)$  factor where  $k$  is the number of elements counted/removed. In particular, it is *not* efficient to count the number of copies of a value in a multiset using the `count` function.

### 8.3.3. Priority Queues

Insertion and removal take  $O(\log n)$  time, and retrieval takes  $O(1)$  time.

If we only need to efficiently find minimum or maximum elements, it is a good idea to use a priority queue instead of a set or multiset.

By default, the elements in a C++ priority queue are sorted in decreasing order, and it is possible to find and remove the largest element in the queue.

```
1 priority_queue<int> q;
2 q.push(3);
3 q.push(5);
4 q.push(7);
5 q.push(2);
```

```

6  cout << q.top() << "\n"; // 7
7  q.pop();
8  cout << q.top() << "\n"; // 5
9  q.pop();
10 q.push(6);
11 cout << q.top() << "\n"; // 6
12 q.pop();

```

If we want to create a priority queue that supports finding and removing the smallest element, we can do it as follows:

```

1  priority_queue<int, vector<int>, greater<int>> q;

```

### 8.3.4. Policy-based data structures

The g++ compiler also supports some data structures that are not part of the C++ standard library. Such structures are called policy-based data structures. To use these structures, the following lines must be added to the code:

```

1  #include <ext/pb_ds/assoc_container.hpp>
2  using namespace __gnu_pbds;

```

After this, we can define a data structure `indexed_set` that is like set but can be indexed like an array. The definition for int values is as follows:

```

1  using indexed_set = tree<
2      int,
3      null_type,
4      less<int>,
5      rb_tree_tag,
6      tree_order_statistics_node_update
7  >;

```

This data structure supports all the operations as a set in C++ in addition to the following:

- `find_by_order(k)`: returns an iterator to the  $k$ -th smallest element (0-based indexing)
- `order_of_key(x)`: returns the number of elements in the set that are strictly less than  $x$